

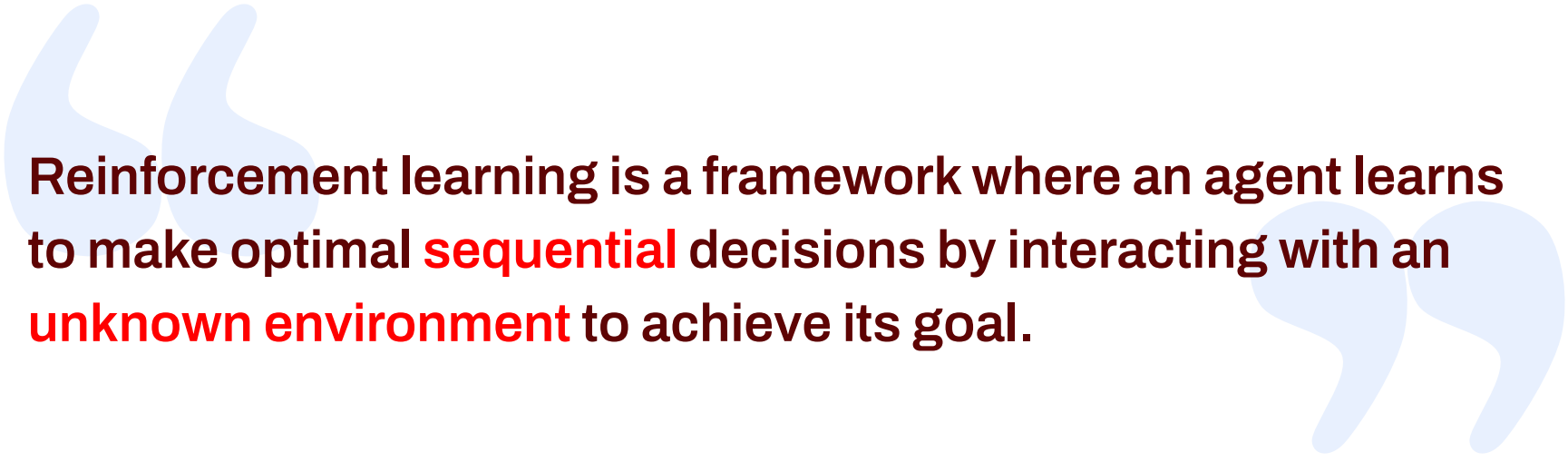
(Re-)Introduction to Reinforcement Learning

Suei-Wen Chen

Weekend 4, From Data to Solutions
Summer 2026



What is RL? One-sentence Summary



Reinforcement learning is a framework where an agent learns to make optimal **sequential** decisions by interacting with an **unknown environment** to achieve its goal.

Roadmap of This Weekend



- 1 Introduction
- 2 Policy Gradient Methods: REINFORCE & Baselines
- 3 Variance Reduction: Actor-Critic
- 4 Credit Assignment: GAE and PPO
- 5 Human Alignment & Reasoning: RL in LLM
- 6 New Paradigm: Safe/Risk-Aware RL
- 7 Project Release

Roadmap of This Lecture

- 1 What can we do with RL?
- 2 Formalism: Markov Decision Processes
- 3 Value functions & Optimal Policies
- 4 Learning with perfect knowledge of the world: Bellman Equations
- 5 Learning without knowledge of the world: Temporal Difference
- 6 Summary

What can we do with RL?

More than games, probably more than you think

Early Major Developments

AlphaGo (2016)



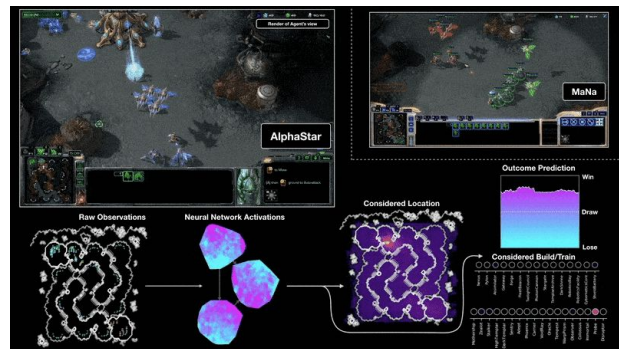
First to defeat a world champion in Go (MCTS)

AlphaZero (2017), MuZero (2019)



From board games to Atari games

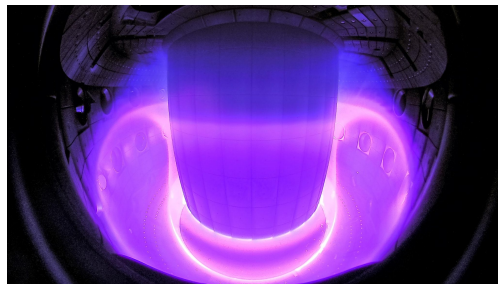
AlphaStar (2019)



StarCraft II (real-time, imperfect information, multi-agent)

Thermonuclear Fusion Control (2022)

EPFL Swiss Plasma Center / DeepMind



Beyond “Games”

AlphaTensor (2022)

Matrix multiplication

$$\begin{bmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \\ 5 & -1 & 2 \end{bmatrix} \times \begin{bmatrix} 2 & -1 & 0 \\ 5 & 1 & -1 \\ -2 & 0 & 0 \end{bmatrix} = \begin{bmatrix} -2 & -1 & 0 \\ 11 & -2 & -1 \\ 1 & -6 & 1 \end{bmatrix}$$
$$3 \times 2 + 1 \times 5 + 0 \times -2 = 11$$

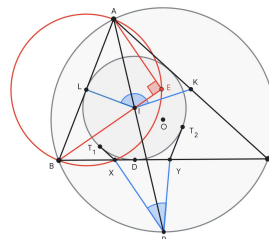
Delivery Robots

RIVR (2026)



AlphaProof (2024),

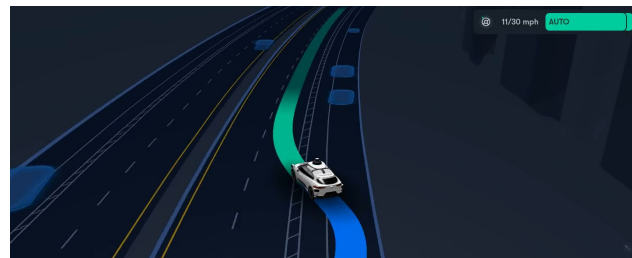
AlphaGeometry (2024, 2025)



AlphaProof Nexus (May 2026)

Autonomous Cars

Waymo World Model (2026)



Modern LLMs

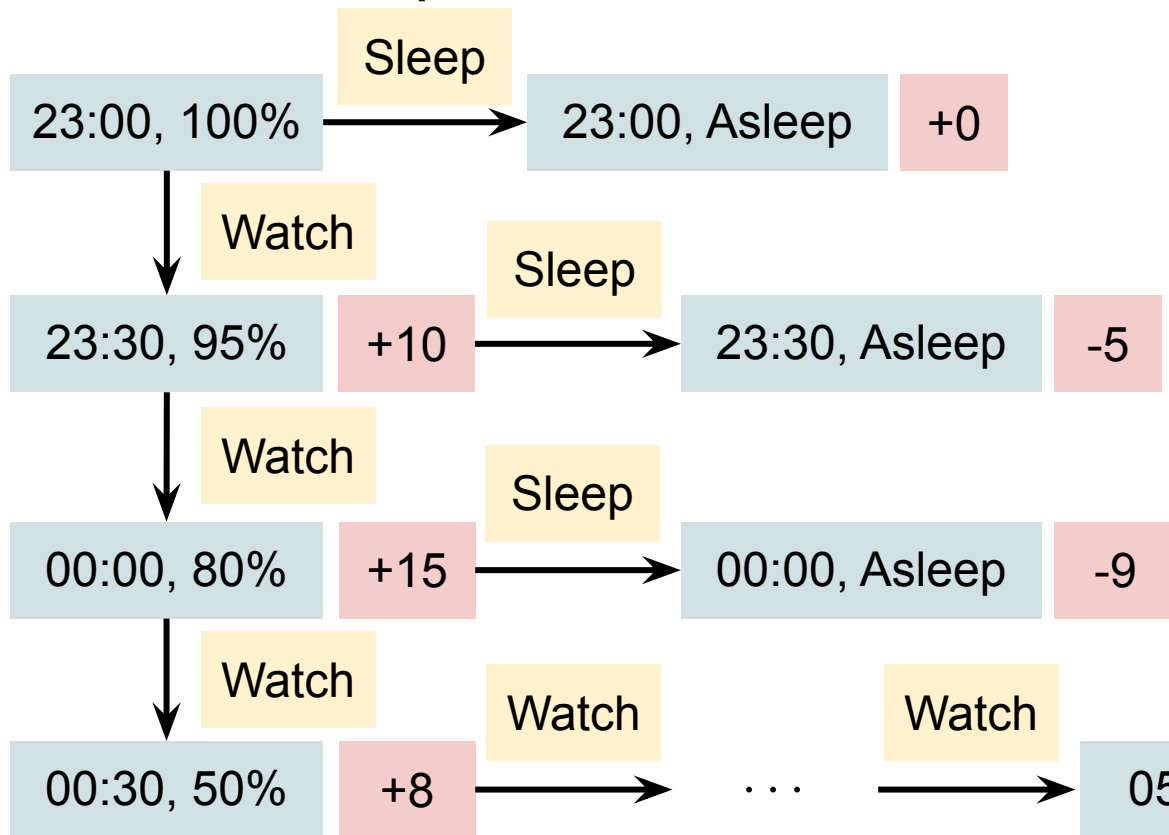


Alignment, Reasoning, Tooling

Markov Decision Processes (MDPs)

Framework for (memoryless) sequential decision making

One More Episode?



Time, Energy



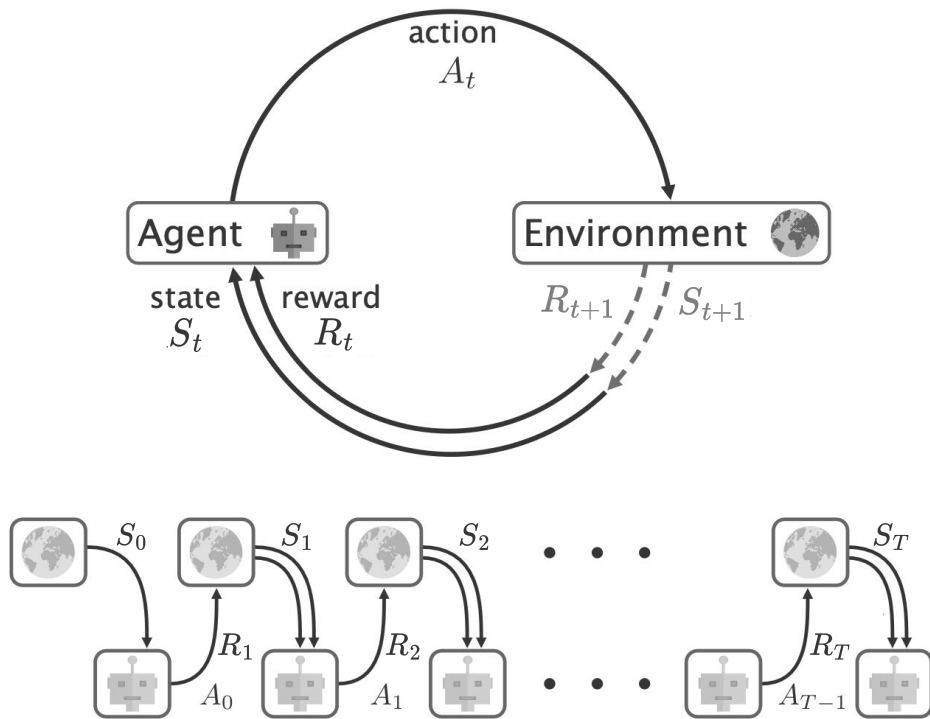
Action



Satisfaction gain

How to act in order to maximize cumulative satisfaction?

MDP: Agent-Environment Interface



At time $t = 0, 1, 2, \dots$ the agent

- observes **state** S_t
- takes **action** A_t
- receives **reward** R_{t+1}
- transitions to a new state S_{t+1}

The agent achieves its goal by maximizing the **expected return** G (cumulative reward) in the long run

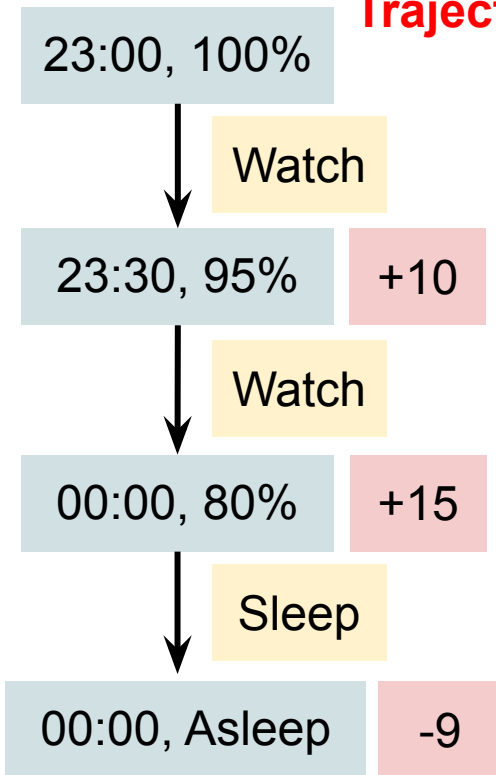
The randomness in the environment is formulated as:

Model / Transition Dynamics

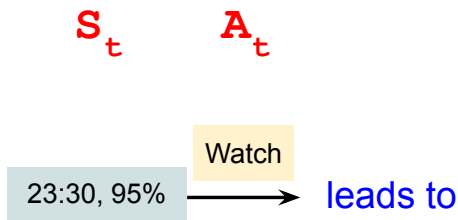
$$S_{t+1}, R_{t+1} \sim P(\cdot, \cdot | S_t, A_t)$$

One More Episode?

Trajectory



Transition:



State s



Time, Energy

Action a



Action

Reward R



Satisfaction gain

S_{t+1} R_{t+1}

00:00, 80%	+8
00:00, 88%	+6
00:00, 70%	+12
⋮	

$P(S_{t+1}, R_{t+1} | S_t, A_t)$

with probability 0.2

with probability 0.4

with probability 0.1

⋮

Return $G = 10 + 15 + (-9)$

Policy: How to act in each scenario

A policy tells you how likely an action will be taken next given the current state:

Deterministic $\pi : s \mapsto \pi(s)$	$\pi (\text{ 23:30, 95\% }) = \text{ Sleep }$
	$\pi (\text{ 01:00, 55\% }) = \text{ Watch }$
Stochastic $\pi : s \mapsto \pi(\cdot s)$	$\pi (\text{ Watch } \text{ 01:00, 55\% }) = 0.6$
	$\pi (\text{ Sleep } \text{ 01:00, 55\% }) = 0.4$

Idea: Evaluate a policy based on returns from the resulting trajectories

Return: Gains in the Long Run

Last night you watched several episodes and experienced the trajectory

$$\tau : S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

The return $\mathbf{G}$ is the (discounted) cumulative rewards:

$$G = R_1 + \gamma R_2 + \gamma R_3 + \dots$$

$\gamma \in [0, 1]$ is called the discount factor (economics, mathematical convenience)

Since $\mathbf{G}$ is random, we take its expectation $\mathbb{E}[G]$ as the performance measure

Value Functions & Optimal Policies

Evaluate & improve policies to find “the best one”

Value Functions: How good a policy is

state-value function $V^\pi(s)$

Tells you how good **state** s is under π

$$V^\pi(s) = \mathbb{E}[G | S_0 = s]$$

(expected return starting at s following π)

action-value function $Q^\pi(s, a)$

Tells you how good **action** a is at state s under π

$$Q^\pi(s, a) = \mathbb{E}[G | S_0 = s, A_0 = a]$$

(expected return starting at s , first taking action a then following π)

$V^\pi(01:00, 55\%)$ = long-term satisfaction starting at $01:00, 55\%$ and following π

$Q^\pi(01:00, 55\%, \text{Watch})$ = long-term satisfaction starting at $01:00, 55\%$, first doing **Watch** and then following π

Value Functions: How good a policy is

state-value function $V^\pi(s)$

Tells you how good **state** s is under π

$$V^\pi(s) = \mathbb{E}[G | S_0 = s]$$

(expected return starting at s following π)

action-value function $Q^\pi(s, a)$

Tells you how good **action** a is at state s under π

$$Q^\pi(s, a) = \mathbb{E}[G | S_0 = s, A_0 = a]$$

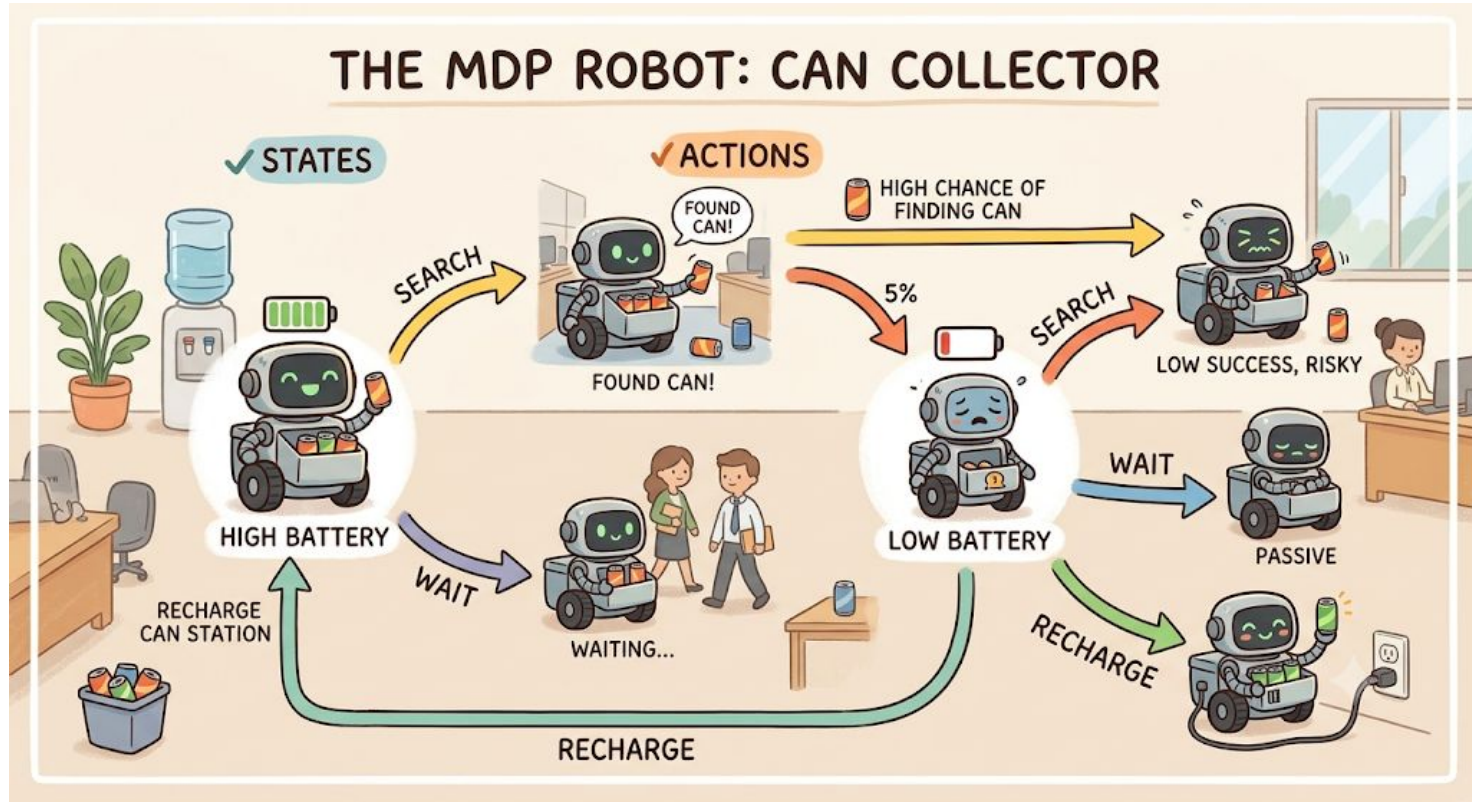
(expected return starting at s , first taking action a then following π)

Goal of RL: find an optimal policy (best under any “scenario”)

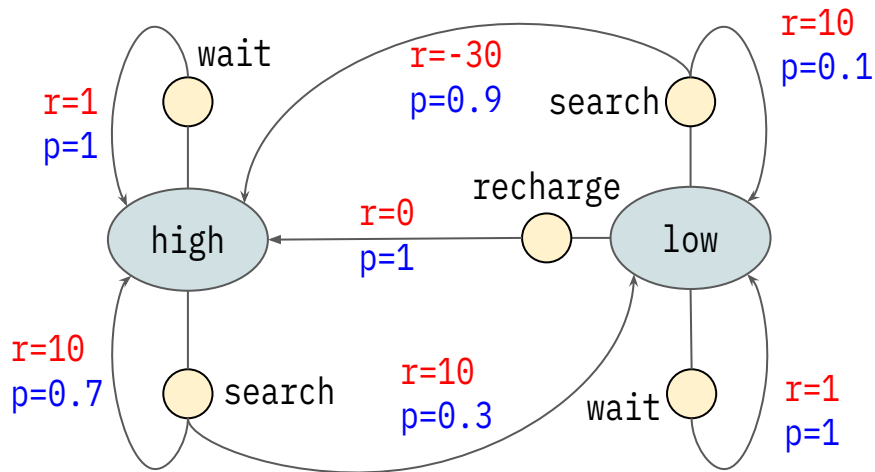
Optimal Policy: a policy π_* with the highest possible values of V^π (or Q^π)

How can we obtain an optimal policy (if we know the value functions)?

Example: Can-Collecting Robot



Example: Can-Collecting Robot



Current policy π

$\pi(a s)$	wait	search	recharge
high	0.1	0.9	0
low	0.5	0.1	0.4

Trajectory τ

low, recharge, 0, high, search, 10, high, search, 10, low, search, -30, high, ...

Return G

$$G = 0 + 0.9 \times 10 + 0.9^2 \times 10 + 0.9^3 \times (-30) + \dots$$

$V^\pi(\text{low})$ = expected # of cans collected in the long run starting from low and following π

$Q^\pi(\text{low}, \text{wait})$ = expected # of cans collected in the long run starting from low, taking wait and following π

Example: Recycling 🤖 ($\gamma=0.9$)

Larger table value = Better policy

Greedy Policy w.r.t. Q

$$\pi_Q(s) := \arg \max_a Q(s, a)$$

(best actions you can take knowing Q)

$\pi(a s)$	wait	search	recharge
high	0.1	0.9	0
low	0.5	0.1	0.4

s	$V^\pi(s)$
high	56.3
low	42.0

$Q^\pi(s, a)$	wait	search	recharge
high	51.7	56.8	x
low	38.8	23.4	50.6

$\pi'(a s)$	wait	search	recharge
high	0	1	0
low	0	0	1

s	$V^{\pi'}(s)$
high	78.7
low	70.9

$Q^{\pi'}(s, a)$	wait	search	recharge
high	71.9	78.7	x
low	64.8	44.2	70.9

Optimal value functions: value functions of an optimal policy

What we've done: Two Central Recipes in RL

1. **Policy Evaluation:** Compute the table
2. **Policy Improvement:** Compute the greedy policy from the table

Policy Improvement Theorem

If the current policy is π , then the greedy policy w.r.t. Q^π is at least as good as π .

Policy Iteration Algorithm (PI)

Starting from any initial policy, alternating between *policy evaluation* (E) and *policy improvement* (I) yields a sequence of improving policies:

$$\pi_0 \xrightarrow{\text{E}} Q^{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} Q^{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} Q^{\pi_*}$$

Two Central Recipes in RL: Neither can be done exactly!

1. **Policy Evaluation:** Compute the table
2. **Policy Improvement:** Compute the greedy policy from the table

Issues

- (E)(I) Curse of dimensionality (too many states/actions)
- (E) Unknown transition dynamics
- (I) Exploration-Exploitation Dilemma (greedy policy allows for no exploration)

Generalized Policy Iteration (GPI)

Interleave steps approximating (E) and (I) in any way that may lead to progress

***Almost every RL algorithm is an instantiation/execution of GPI
(e.g. Actor-Critic, PPO, ...)***

Bellman Equations

Compute the tables, assuming perfect knowledge of the world

Bellman Equation: Structural Equations of MDPs

Q: How can we compute value functions for a given policy?

Observation: It's complicated. Value at one state/action depends on the future

Idea: Break down the entire future via *one-step lookahead*



Value Now = Immediate Reward + Discounted Value @ Next Step

This apply to both the state-value function V^π and the action-value function Q^π

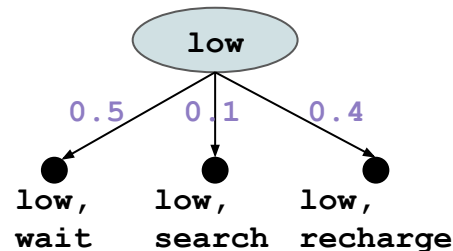
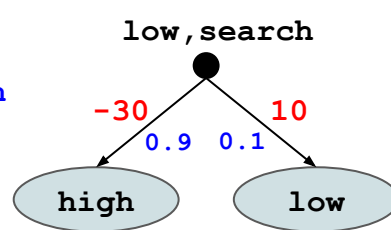
The One-Step Lookahead Principle

$\pi(a s)$	wait	search	recharge
high	0.1	0.9	0
low	0.5	0.1	0.4

immediate
rewards

transition
probs

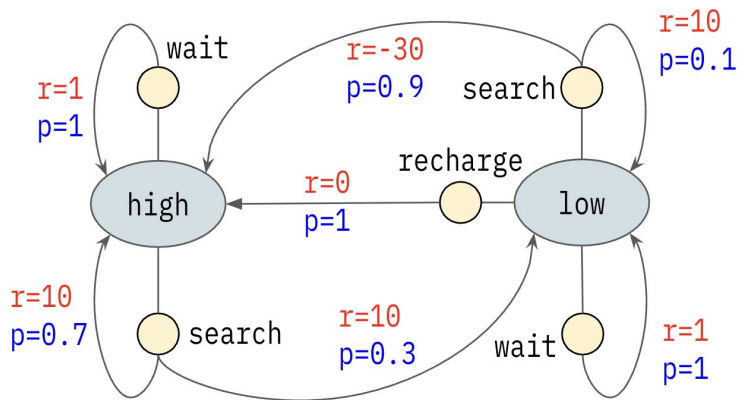
$\pi(a|s)$



$$Q^{\pi}(\text{low}, \text{search}) = 0.9[-30 + \gamma V^{\pi}(\text{high})] + 0.1[10 + \gamma V^{\pi}(\text{low})]$$

$$V^{\pi}(\text{low}) = 0.5 \times Q^{\pi}(\text{low}, \text{wait}) + 0.1 \times Q^{\pi}(\text{low}, \text{search}) + 0.4 \times Q^{\pi}(\text{low}, \text{recharge})$$

Observe: q^{π} and v^{π} are swapped on RHS



For the curious: Same idea, in symbols

long-term payoff starting at state s , taking action a and then following policy π

immediate payoff after taking action a at state s

discounted long-term payoff at the new state s'

$$Q^\pi(s, a) = \mathbb{E}_{r, s'}^\pi [r + \gamma V^\pi(s')] = \mathbb{E}_{r, s'}^\pi [r + \gamma \mathbb{E}_{a'}^\pi [Q^\pi(s', a')]] = \mathbb{E}_{r, s', a'}^\pi [r + \gamma Q^\pi(s', a')]$$

average over all possibilities using transition dynamics & policy

substitute

combine expectations

$$V^\pi(s) = \mathbb{E}_a^\pi [Q^\pi(s, a)]$$

$$= \mathbb{E}_a^\pi [\mathbb{E}_{r, s'}^\pi [r + \gamma V^\pi(s')]]$$

$$= \mathbb{E}_{a, r, s'}^\pi [r + \gamma V^\pi(s')]$$

long-term payoff starting at state s and then following policy π

choose action a from policy π , then observe its Q value

Value Now = Immediate Reward + Discounted Value @ Next Step, on average

Solving BEs gives us the tables

Value Now

Immediate
Reward

Discounted Value
@ Next Step

$$Q^{\pi}(s, a) = \mathbb{E}_{r, s', a'}^{\pi} [r + \gamma Q^{\pi}(s', a')]$$

$$V^{\pi}(s) = \mathbb{E}_{a, r, s'}^{\pi} [r + \gamma V^{\pi}(s')]$$

If we know the **transition dynamics**, we can solve these equations for the table.

Issue: We don't know the environment, so we cannot compute the **averages**!

→ ***We need to learn from experience, by interacting with the environment***

Temporal Difference (TD) Learning

Compute the tables, without knowledge of the world

Challenge: Compute the V-table for a policy π

What we know: In theory, V satisfies the Bellman Equation

$$V^\pi(s) = \mathbb{E}^\pi[r + \gamma V^\pi(s')]$$

What we don't know: The transition rules and probabilities

What we can do: Let our actor take a step in the environment



Insight: We can estimate $v^\pi(s)$ using $r + \gamma v^\pi(s')$ since they are equal on average

Circular Issue: To estimate v^π we need to know v^π first ...

Robbins-Monro: Shooting darts in the dark

Goal: Hit a hidden **bullseye, value of $V(s)$** , on a wall in a pitch-black room

Rule: You could only guess where it is by throwing a dart repeatedly

- At each round, you aim your shot at **your current best guess, $\hat{V}(s)$**
- A referee gives you feedback about your throw, from which you infer **a new best guess**

Strategy: nudge your next aim towards **the inferred target (new best guess)**

$$\text{Next Aim} = \text{Current Aim} + \text{Learning Rate} \times (\text{Inferred Target} - \text{Current Aim})$$



Temporal Difference (TD) Learning

1. Start with a random guess $\hat{V}(s)$ for $V^\pi(s)$

Shoot the dart once from s

→ referee tells you $\mathbf{r}, s'$

current guess

$$V^\pi(s) \approx \cancel{\mathbb{E}^\pi} \left[r + \gamma \underbrace{\hat{V}(s')}_{\text{new educated guess on } \text{🎯} \text{ location}} \right]$$

🎯 hidden target

2. Update rule: From s , take one action following π , observe $\mathbf{r}, s'$ and compute

$$\hat{V}(s) \leftarrow \hat{V}(s) + \alpha_t \left[r + \gamma \hat{V}(s') - \hat{V}(s) \right]$$

Next Aim = Current Aim + Learning Rate × (Inferred Target - Current Aim)

TD targets and TD errors

Next Aim = Current Aim + Learning Rate × (Inferred Target - Current Aim)

$$\hat{V}(s) \leftarrow \hat{V}(s) + \alpha_t \left[r + \gamma \hat{V}(s') - \hat{V}(s) \right]$$

- $r + \gamma \hat{V}(s')$ is called the (one-step) **TD target**
- $r + \gamma \hat{V}(s') - \hat{V}(s)$ is called the (one-step) **TD error**
- It's possible to take multiple steps into the future! This is called n-step TD (related to *bias-variance tradeoff*, more on this in the afternoon)

Beyond Simplistic Environments

TD-Learning Learn the V^π table via $\hat{V}(s) \leftarrow \hat{V}(s) + \alpha_t [r + \gamma \hat{V}(s') - \hat{V}(s)]$

Most environments of interest have too many states to fit in a table!

- Chess: $\sim 10^{44}$ possible board positions
- Autonomous cars: infinite state space (camera image, sensor readings ...)

Idea: Approximate the V^π table using a *learnable function* V_θ

$$V^\pi(s) \approx V_\theta(s)$$

For example, V_θ could be a neural network with parameters θ .

Extend TD Learning to Neural Networks

A NN V_θ is trained via **stochastic gradient descent** on a **loss function**.

We want to drive the **current prediction** $V_\theta(s)$ towards the **target** $r + \gamma V_\theta(s')$.

→ We can view this as a regression problem!

A natural loss function is the **(mean) squared error loss**:

$$L(\theta) = \frac{1}{2} (r + \gamma V_\theta(s') - V_\theta(s))^2$$

Learning: Update θ to nudge $V_\theta(s)$ towards $r + \gamma V_\theta(s')$ by performing SGD:

$$\theta \leftarrow \theta - \alpha_t \nabla_\theta L(\theta)$$

Summary

In this lecture, we discussed ...

- The agent-environment interface of RL, formalized as MDP
- What a policy is, and how to assess their quality using value functions
- The one-step-lookahead principle and the structural (Bellman) equations
- How to evaluate a policy by trial and error, using the TD target
- How to extend TD learning from a table to a neural network